Battery State-of-Health Prediction Pipeline

End-to-end ML pipeline for lithium-ion battery health classification using CALCE CS2 cycling data

---

Rohit
April 2026

Tech Stack: Python, Pandas, Scikit-learn, XGBoost, MLflow, Plotly Dash, Matplotlib

1. Project Overview

This project implements a complete machine learning pipeline that predicts lithium-ion battery health tiers from real-world cycling data. It ingests raw charge/discharge measurements from the University of Maryland CALCE battery research group, engineers degradation-aware features at the cycle level, trains an XGBoost classifier that achieves 99.5% accuracy on held-out batteries, and surfaces all results through an interactive Plotly Dash dashboard. The pipeline is designed for fleet-level battery management: identifying which batteries need monitoring, which need replacement, and how much useful life remains in each cell.

2. Dataset Description

Source: CALCE Battery Research Group, University of Maryland (https://calce.umd.edu/battery-data). The CS2 dataset contains cycling data from six prismatic lithium-ion cells (CS2_33 through CS2_38) with a nominal capacity of 1.1 Ah.

Size: 5,515 charge/discharge cycles across 6 cells. Each cycle records charge capacity, discharge capacity, voltage extremes, charge/discharge times, and internal resistance. Cells were cycled between 753 and 1,051 times under controlled lab conditions until significant degradation occurred.

Preprocessing: Raw Arbin Excel files are loaded per cell, cumulative capacity counters are differenced to obtain per-cycle discharge capacity, and SOH is computed as the ratio of current capacity to the initial capacity (average of the first three cycles). Health tiers are assigned as: Good (SOH >= 90%), Moderate (80-90%), Weak (< 80%).

Battery Fleet Status — Latest Cycle

Battery	SOH (%)	Predicted	Actual	Cycles	Fade Rate
CS2_33	76.63	Weak	Weak	843	-0.01
CS2_34	81.82	Moderate	Moderate	753	0
CS2_35	87.78	Moderate	Moderate	908	0
CS2_36	9.6	Weak	Weak	948	-0.01
CS2_37	6.36	Weak	Weak	1012	-0.01
CS2_38	6.37	Weak	Weak	1051	-0.01

Figure 1: Fleet status at each battery's final cycle. Four of six cells reached the Weak tier, with CS2_36/37/38 experiencing near-total capacity collapse.

3. Project Architecture

The pipeline consists of four sequential stages, each implemented as a standalone Python module. All experiment parameters, metrics, and model artifacts are logged to MLflow for reproducibility.

Stage	Module	Output
1. Data Ingestion	data_loader.py	calce_battery_data.csv (5,515 rows)
2. Feature Engineering	feature_engineering.py	engineered_features.csv (5,455 rows)
3. Classification	train_classifiers.py	XGBoost (99.5%) + LogReg (94.7%)

4. Clustering	clustering.py	K-Means (k=3, ARI=0.32)

Feature engineering produces one row per cycle (after a 10-cycle warmup window), using rolling-window statistics to capture degradation trends. Nine features are computed: capacity fade rate, resistance growth rate, charge/discharge ratio, voltage delta trend, cycle count, current discharge capacity, current SOH, capacity variance, and energy efficiency.

The classifier uses group-aware train/test splitting (GroupShuffleSplit) to ensure all cycles from one battery stay together. Training set: CS2_35/36/37/38. Test set: CS2_33/34. This prevents data leakage between batteries and tests whether the model generalizes to entirely unseen cells.

4. Dashboard Walkthrough

The interactive dashboard is built with Plotly Dash and organized into four sections on a single scrollable page. Each section answers a specific question about the battery fleet.

4.1 Fleet KPIs — "How is the fleet doing at a glance?"

Six cards summarize the fleet: total batteries tracked (6), count per health tier (0 Good, 2 Moderate, 4 Weak), the fleet-wide average SOH (44.8%), and a color-coded gauge. The red gauge immediately signals that the fleet average is well below the 80% Weak threshold — this is expected because four of six cells were cycled to near-failure in the CALCE aging study.

4.2 SOH Distribution — "Where are cycles concentrated?"

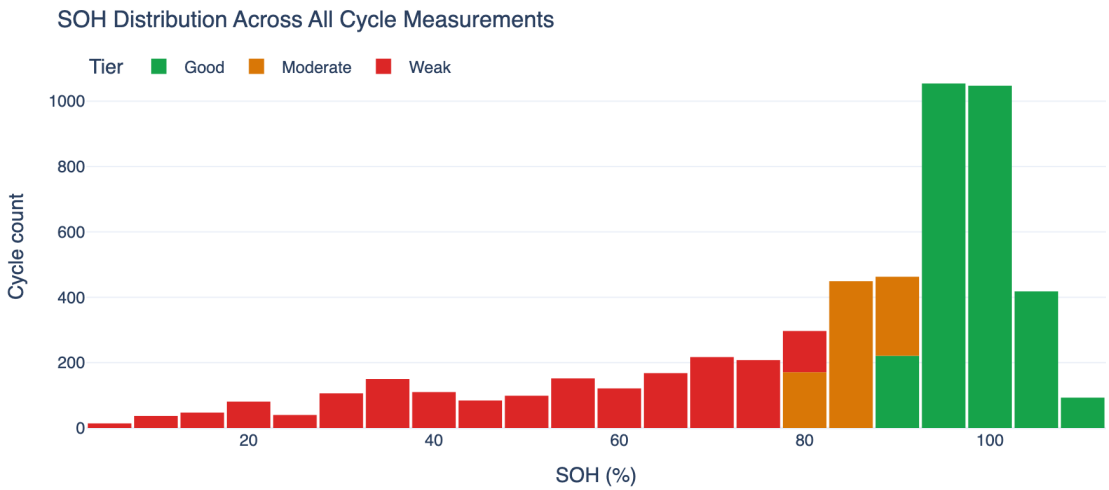


Figure 2: SOH distribution across all 5,515 cycle measurements. The tall green bars near 100% represent the early healthy life of each battery. The red left tail shows the degradation phase.

This histogram counts every individual cycle measurement, not just the final state. Since every battery starts life near 100% SOH, the large green cluster on the right represents hundreds of early-life cycles. The spread of red bars from 0-80% shows how batteries CS2_36/37/38 experienced catastrophic capacity loss in their later cycles. The amber Moderate band (80-90%) is comparatively narrow, indicating that batteries transition through this zone relatively quickly.

4.3 Battery Explorer — "What is happening to each battery?"

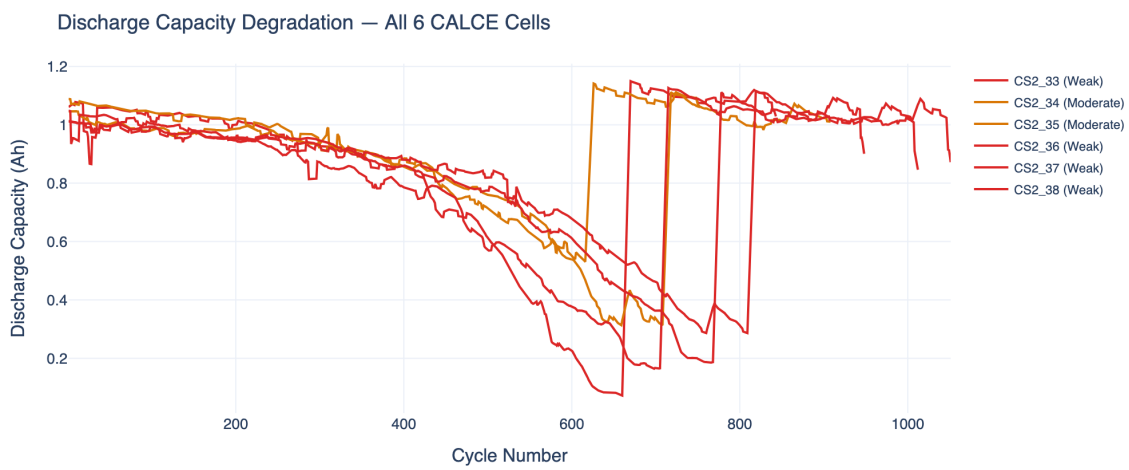


Figure 3: Smoothed discharge capacity curves for all six cells. Lines are colored by final health tier. CS2_36/37/38 show sharp capacity collapse after ~600 cycles.

Each line traces a battery's discharge capacity (in Ah) across its lifetime. CS2_33/34/35 show gradual, steady degradation — the kind of aging pattern that is predictable and manageable. CS2_36/37/38 exhibit a sharp "knee" around cycle 600-700 where capacity collapses from ~0.8 Ah to near zero. This non-linear failure mode is characteristic of real-world lithium-ion batteries and is one reason why SOH forecasting is critical: by the time the knee is visible, the battery is already approaching end-of-life. The dashboard allows users to select individual batteries and immediately see both the degradation curve and a remaining-useful-life forecast.

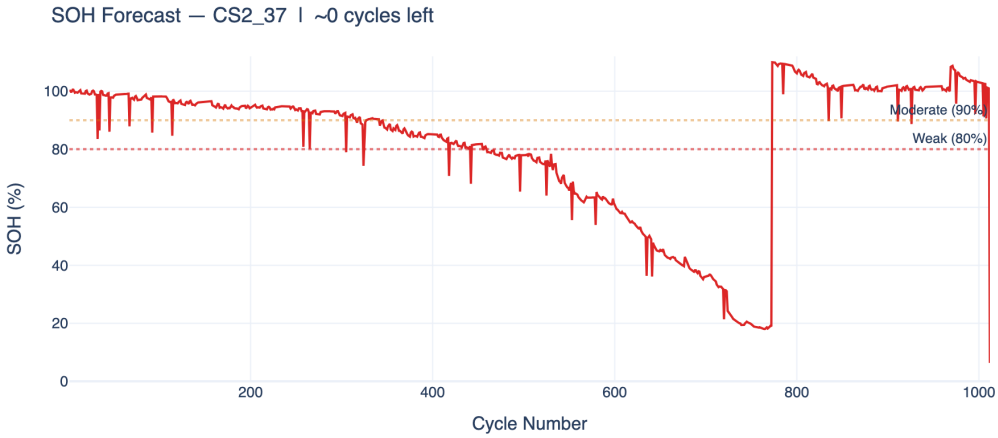


Figure 4: SOH forecast for CS2_37 showing historical trajectory and linear projection. This battery crossed the Weak threshold around cycle 600 and has no remaining useful life.

5. Key Results

Model	Metric	Value	Notes
XGBoost	Test Accuracy	99.5%	40-iter RandomizedSearchCV, 5-fold CV
XGBoost	F1 (macro)	0.99	Near-perfect per-class precision/recall
Logistic Regression	Test Accuracy	94.7%	Baseline comparison
K-Means (k=3)	Adjusted Rand Index	0.32	Unsupervised, no label access

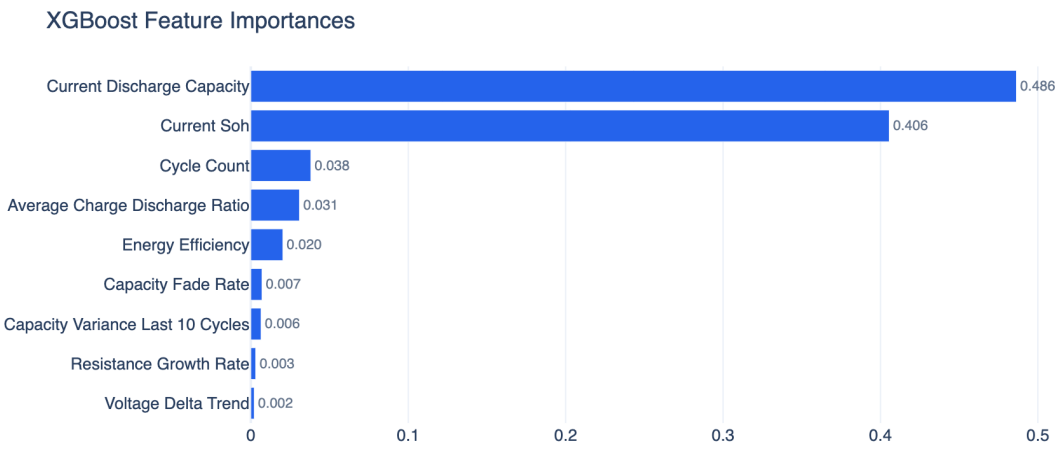
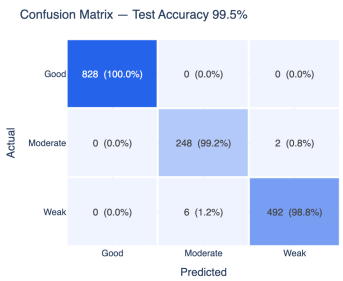


Figure 5: XGBoost feature importances. Current discharge capacity and SOH are by far the strongest predictors, accounting for nearly 90% of importance combined.

The XGBoost classifier achieves 99.5% accuracy on held-out batteries CS2_33 and CS2_34 (1,576 cycle-level predictions). Feature importance analysis reveals that current discharge capacity (0.486) and current SOH (0.406) dominate the model's decisions — together accounting for ~89% of total importance. This makes physical sense: these features directly measure how much energy the battery can still deliver. Secondary features like cycle count, charge/discharge ratio, and energy efficiency contribute marginally, suggesting that the model relies primarily on direct capacity measurements rather than derived trends.



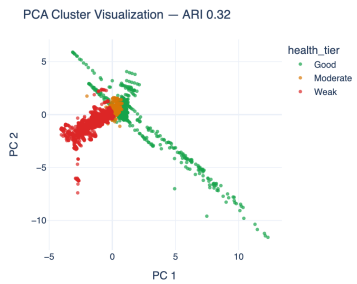


Figure 6 (left): Confusion matrix on the test set. Only 8 out of 1,576 predictions were incorrect — all near the Moderate/Weak boundary. Figure 7 (right): PCA visualization colored by actual health tier, showing clear separation between Good (green) and Weak (red) clusters.

6. How to Run

The project requires Python 3.10+ and the dependencies listed in requirements.txt.

```
# Clone and enter the project
cd battery soh project

# Install dependencies
pip install -r requirements.txt

# Run the full pipeline (data loading, features, models, clusters)
python run pipeline.py

# Launch the interactive dashboard
python src/dashboard.py
# Open http://localhost:8050

# (Optional) View MLflow experiment logs
mlflow ui --backend-store-uri mlruns
# Open http://localhost:5000
```

7. Tech Stack

Component	Technology
Data Processing	Python 3.11, Pandas, NumPy
Machine Learning	Scikit-learn, XGBoost
Experiment Tracking	MLflow
Dashboard	Plotly, Dash, Dash Bootstrap Components
Static Plots	Matplotlib
Data Source	CALCE CS2 (University of Maryland)

8. References

[1] CALCE Battery Research Group, University of Maryland. CS2 Battery Dataset. https://calce.umd.edu/battery-data

[2] Chen, T. & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of KDD 2016.

[3] Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python. JMLR 12, pp. 2825-2830.

[4] MLflow: An open-source platform for the machine learning lifecycle. https://mlflow.org